

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import statsmodels.formula.api as smf
from sklearn.linear_model import LogisticRegression
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
QuadraticDiscriminantAnalysis
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, accuracy_score
from sklearn.preprocessing import StandardScaler
df=pd.read_csv(r"D:\ML&Fintech\data_ML&Fintech\Weekly.csv")
df.head()
```

	Year	Lag1	Lag2	Lag3	Lag4	Lag5	Volume	Today	Direction
0	1990	0.816	1.572	-3.936	-0.229	-3.484	0.154976	-0.270	Down
1	1990	-0.270	0.816	1.572	-3.936	-0.229	0.148574	-2.576	Down
2	1990	-2.576	-0.270	0.816	1.572	-3.936	0.159837	3.514	Up
3	1990	3.514	-2.576	-0.270	0.816	1.572	0.161630	0.712	Up
4	1990	0.712	3.514	-2.576	-0.270	0.816	0.153728	1.178	Up

```
df.shape
(1089, 9)
df.describe()
```

[illegible]

	Year	Lag1	Lag2	Lag3	Lag4	Lag5	Volume	Today
mean	2000.04 8669	0.15058 5	0.15107 9	0.14720 5	0.14581 8	0.13989 3	1.57461 8	0.14989 9
std	6.03318 2	2.35701 3	2.35725 4	2.36050 2	2.36027 9	2.36128 5	1.68663 6	2.35692 7
min	1990.00 0000	- 18.1950 00	- 18.1950 00	- 18.1950 00	- 18.1950 00	- 18.1950 00	0.08746 5	- 18.1950 00
25%	1995.00 0000	- 1.15400 0	- 1.15400 0	- 1.15800 0	- 1.15800 0	- 1.16600 0	0.33202 2	- 1.15400 0
50%	2000.00 0000	0.24100 0	0.24100 0	0.24100 0	0.23800 0	0.23400 0	1.00268 0	0.24100 0
75%	2005.00 0000	1.40500 0	1.40900 0	1.40900 0	1.40900 0	1.40500 0	2.05372 7	1.40500 0
max	2010.00 0000	12.0260 00	12.0260 00	12.0260 00	12.0260 00	12.0260 00	9.32821 4	12.0260 00

df.isna().sum()

Year 0

Lag1 0

Lag2 0

Lag3 0

Lag4 0

Lag5 0

Volume 0

Today 0

Direction 0

dtype: int64

df_modified = df.drop('Direction', axis=1)

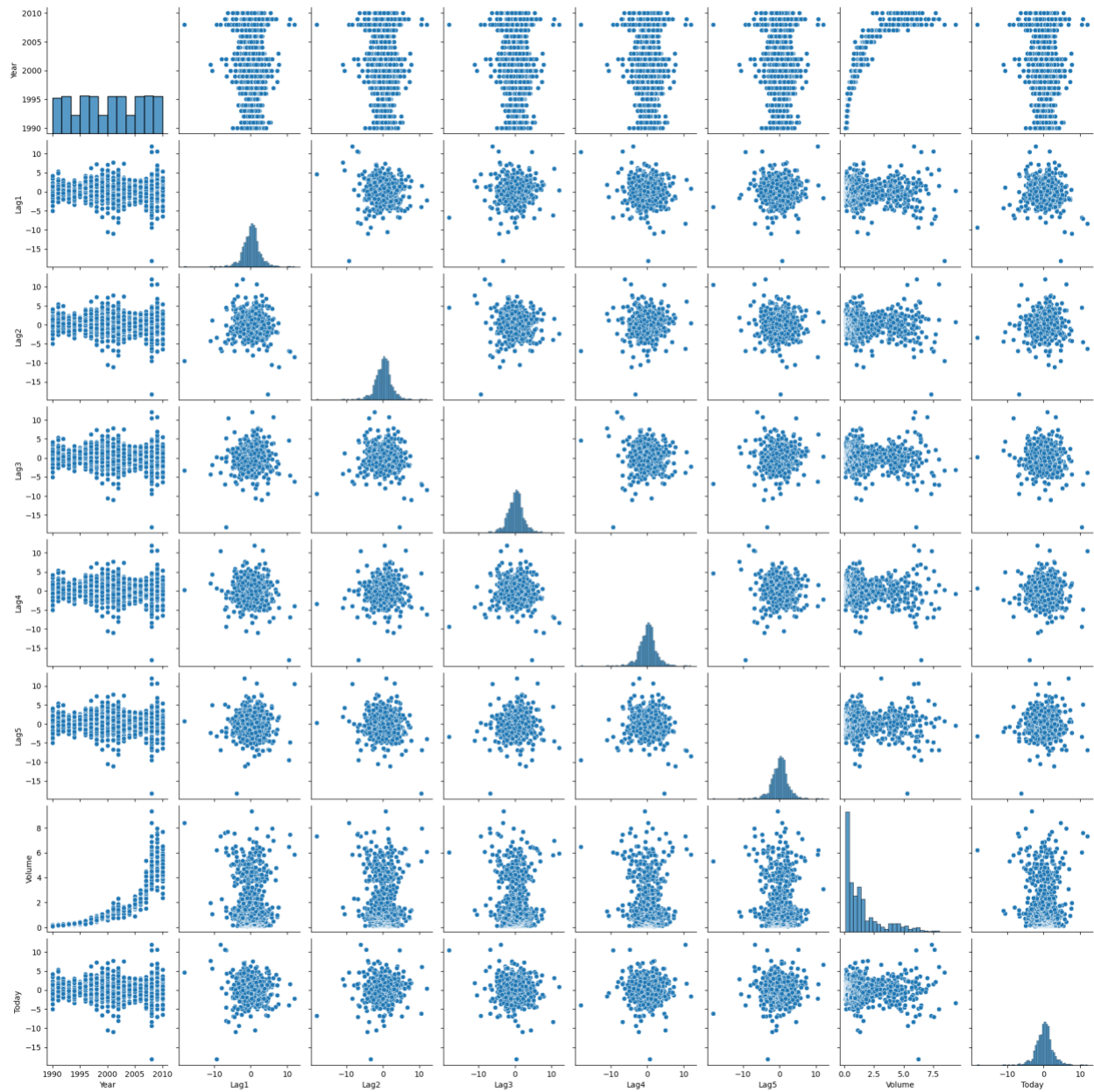
df_modified.corr()

	Year	Lag1	Lag2	Lag3	Lag4	Lag5	Volume	Today
Year	1.00000 0	0.03228 9	0.03339 0	0.03000 6	0.03112 8	0.03051 9	0.84194 2	0.03246 0
Lag1	0.03228 9	1.00000 0	0.07485 3	0.05863 6	0.07127 4	0.00818 3	0.06495 1	0.07503 2
Lag2	0.03339 0	0.07485 3	1.00000 0	0.07572 1	0.05838 2	0.07249 9	0.08551 3	0.05916 7
Lag3	0.03000 6	0.05863 6	0.07572 1	1.00000 0	0.07539 6	0.06065 7	0.06928 8	0.07124 4
Lag4	0.03112 8	0.07127 4	0.05838 2	0.07539 6	1.00000 0	0.07567 5	0.06107 5	0.00782 6
Lag5	0.03051 9	0.00818 3	0.07249 9	0.06065 7	0.07567 5	1.00000 0	0.05851 7	0.01101 3
Volume	0.84194 2	0.06495 1	0.08551 3	0.06928 8	0.06107 5	0.05851 7	1.00000 0	0.03307 8
Today	0.03246 0	0.07503 2	0.05916 7	0.07124 4	0.00782 6	0.01101 3	0.03307 8	1.00000 0

The near-zero correlations between lags and today's return means past price movements do not predict future ones, the high correlation between Year and Volume shows a clear upward trend in market activity.

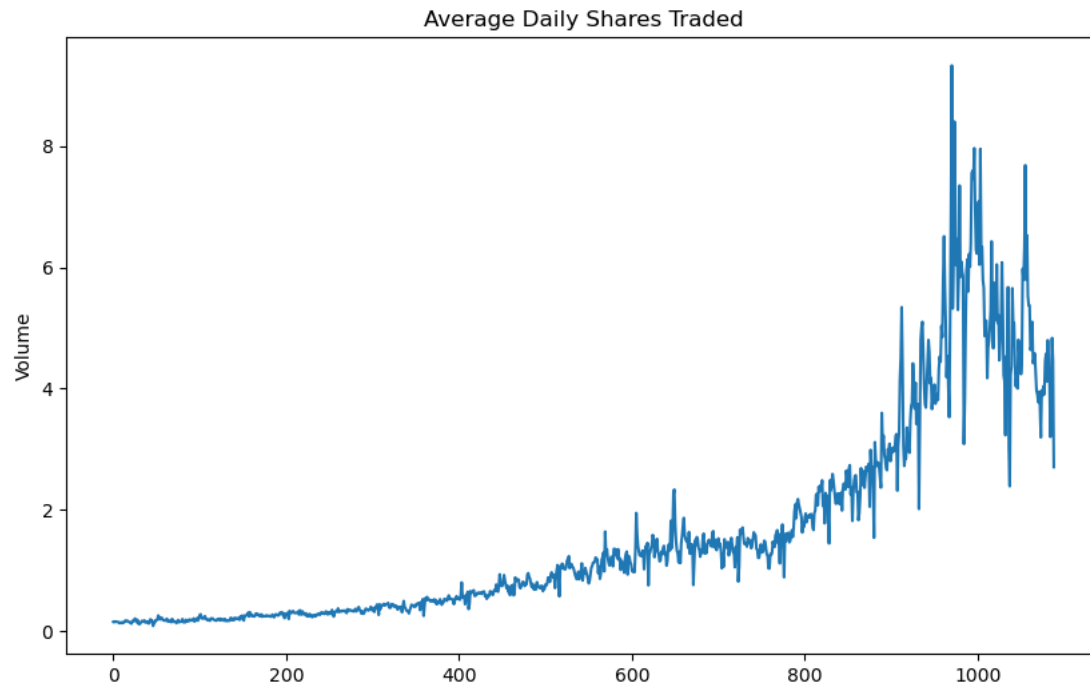
```
sns.pairplot(df)
```

```
<seaborn.axisgrid.PairGrid at 0x1d12b5b27b0>
```



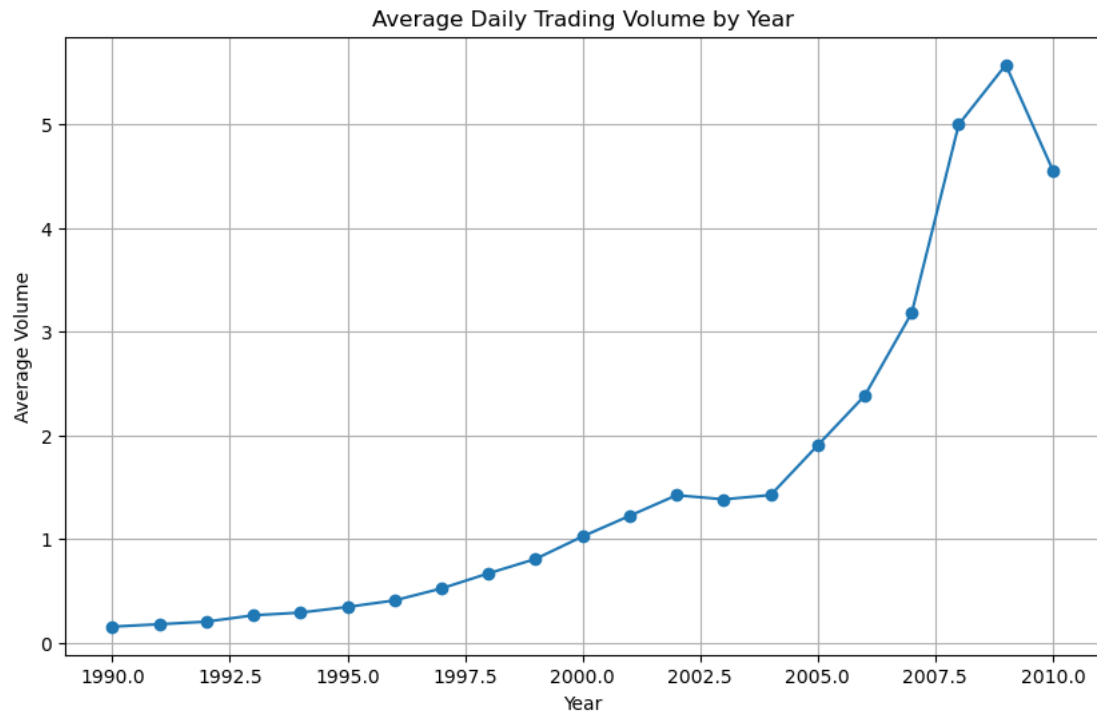
There are no obvious strong relationships between the Lag variables. Only a strong correlation between Volume and Year variables.

```
plt.figure(figsize=(10,6))
plt.plot(df.Volume)
plt.title('Average Daily Shares Traded')
plt.ylabel('Volume')
Text(0, 0.5, 'Volume')
```



```
volume_by_year = df.groupby('Year')['Volume'].mean()
```

```
plt.figure(figsize=(10,6))  
plt.plot(volume_by_year.index, volume_by_year.values, marker='o')  
plt.title('Average Daily Trading Volume by Year')  
plt.xlabel('Year')  
plt.ylabel('Average Volume')  
plt.grid(True)  
plt.show()
```



```
df.Direction.describe()
count    1089
unique     2
top       Up
freq      605
Name: Direction, dtype: object
```

- 'Up': 605
- 'Down': 484

55% percent of the `Direction` data is `Up`, and the other 45% percent is `Down`

(b) Use the full data set to perform a logistic regression with `Direction` as the response and the five lag variables plus `Volume` as predictors. Use the summary function to print the results. Do any of the predictors appear to be statistically significant? If so, which ones?

```
X=df.loc[:, 'Lag1':'Volume']
y = pd.get_dummies(df['Direction'])['Up']

sm_logit = sm.Logit(y, sm.add_constant(X)).fit()
sm_logit.summary()
Optimization terminated successfully.
Current function value: 0.682441
Iterations 4
```

Logit Regression Results

Up No. Observations: 1089

Dep. Variable:**Model:** Logit **Df Residuals:** 1082**Method:** MLE **Df Model:** 6**Date:** Sun, 09 Nov 2025 **Pseudo R-squ.:** 0.006580**Time:** 15:18:59 **Log-Likelihood:** -743.18**converged:** True **LL-Null:** -748.10**Covariance Type:** nonrobust **LLR p-value:** 0.1313

	coef	std err	z	P> z	[0.025	0.975]
const	0.2669	0.086	3.106	0.002	0.098	0.435
Lag1	-0.0413	0.026	-1.563	0.118	-0.093	0.010
Lag2	0.0584	0.027	2.175	0.030	0.006	0.111
Lag3	-0.0161	0.027	-0.602	0.547	-0.068	0.036
Lag4	-0.0278	0.026	-1.050	0.294	-0.080	0.024
Lag5	-0.0145	0.026	-0.549	0.583	-0.066	0.037
Volume	-0.0227	0.037	-0.616	0.538	-0.095	0.050

Lag2 variables are looking statistically significant according to the associated p-values. Other Lag predictors are not statistically significant. This shows weak short-term momentum effect (2-day lag).

Market movements change day to day. R^2) is low and shows past returns barely explain current direction. The overall model does not significantly predict whether the market will go up or down.

```
X = df.loc[:, 'Lag1': 'Volume']  
y = pd.get_dummies(df['Direction'])['Down']
```

```
sm_logit = sm.Logit(y, sm.add_constant(X)).fit()
```

```
sm_logit.summary()
```

Optimization terminated successfully.

Current function value: 0.682441

Iterations 4

Logit Regression Results

Dep. Variable:			Down	No. Observations:	1089	
Model:			Logit	Df Residuals:	1082	
Method:			MLE	Df Model:	6	
Date:	Sun, 09 Nov 2025			Pseudo R-squ.:	0.006580	
Time:	15:19:02			Log-Likelihood:	-743.18	
converged:	True			LL-Null:	-748.10	
Covariance Type:	nonrobust			LLR p-value:	0.1313	
	coef	std err	z	P> z 	[0.025	0.975]
const	-0.2669	0.086	-3.106	0.002	-0.435	-0.098
Lag1	0.0413	0.026	1.563	0.118	-0.010	0.093
Lag2	-0.0584	0.027	-2.175	0.030	-0.111	-0.006
Lag3	0.0161	0.027	0.602	0.547	-0.036	0.068
Lag4	0.0278	0.026	1.050	0.294	-0.024	0.080
Lag5	0.0145	0.026	0.549	0.583	-0.037	0.066
Volume	0.0227	0.037	0.616	0.538	-0.050	0.095

(c) Compute the confusion matrix and overall fraction of correct predictions. Explain what the confusion matrix is telling you about the types of mistakes made by logistic regression.

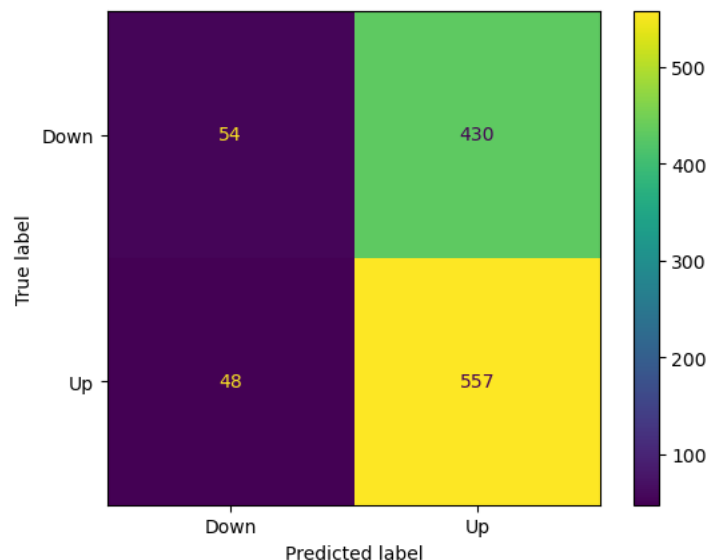
```
X_sk = df.loc[:, 'Lag1':'Volume']
y_sk = df['Direction']
```

```
sk_logit = LogisticRegression(fit_intercept=True)
sk_logit.fit(X_sk, y_sk)
sk_logit.intercept_, sk_logit.coef_
(array([0.26685095]),
 array([[-0.04123352, 0.05839182, -0.01604123, -0.02777908, -0.01445306,
        -0.02273943]]))
acc_score = accuracy_score(y_sk, sk_logit.predict(X_sk))
print(f'Accuracy: {acc_score}')
print(f'Training Error: {1-acc_score}')
Accuracy: 0.5610651974288338
Training Error: 0.43893480257116624
```

By predicting Up every time, we would achieve a baseline accuracy of 55%. However, our logistic regression model only improves the accuracy slightly to 56%. This indicates that the model does not perform significantly better than the baseline, and it may not be suitable for this dataset. The model essentially relying on the intercept term to bias the output towards predicting Up.

```
disp = ConfusionMatrixDisplay(confusion_matrix(y_sk, sk_logit.predict(X_sk)),
display_labels=['Down', 'Up'])
disp.plot()
```

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1d12b6b2ba0>



(d) Now fit the logistic regression model using a training data period from 1990 to 2008, with Lag2 as the only predictor. Compute the confusion matrix and the overall fraction of correct predictions for the held out data (that is, the data from 2009 and 2010).

```
tr = df[df.Year <= 2008]
ts = df[df.Year >= 2009]
```

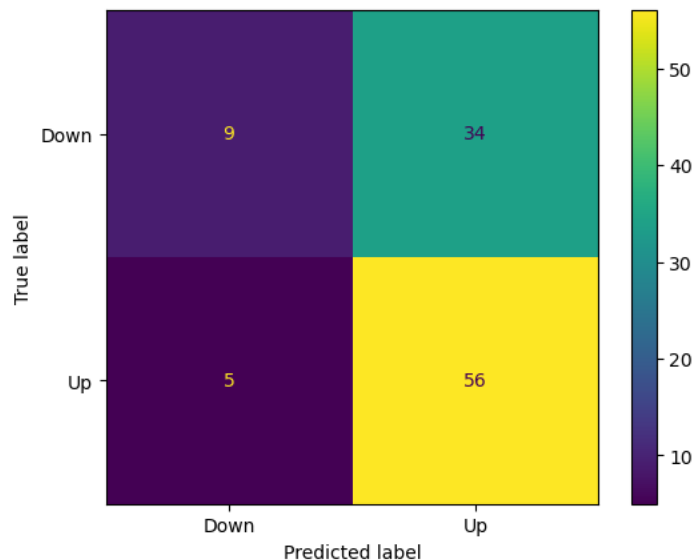
```
X_train = tr.Lag2.values.reshape(-1, 1)
y_train = tr.Direction
```

```
X_test = ts.Lag2.values.reshape(-1, 1)
y_test = ts.Direction
sk_logit_2 = LogisticRegression(fit_intercept=True)
sk_logit_2.fit(X_train, y_train)
```

```
sk_logit_2.intercept_, sk_logit_2.coef_
(array([0.20323048]), array([[0.0580307]]))
acc_score = accuracy_score(y_test, sk_logit_2.predict(X_test))
print(f'Accuracy: {acc_score}')
print(f'Test Error: {1-acc_score}')
Accuracy: 0.625
Test Error: 0.375
```

The accuracy rate of the new model using only `Lag2` as a predictor is 62.5%, which is slightly higher than the baseline accuracy of 55%.

```
disp = ConfusionMatrixDisplay(confusion_matrix(y_test, sk_logit_2.predict(X_test)),
display_labels=['Down', 'Up'])
disp.plot()
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1d130411090>
```



Accuracy (62.5%) is better than random or baseline (55%) , so `Lag2` carries some predictive signal. The model fails to identify “Down” days that is only 9/43 correct (recall $\approx 21\%$). It overpredicts “Up” , likely due to the market being biased upward overall. This imbalance means the model is not balanced — it's biased toward the majority class “Up”. When the market increased 2 days ago, it's slightly more likely to rise today.

Repeat (d) using LDA.

```
weekly_lda = LinearDiscriminantAnalysis()
weekly_lda.fit(X_train, y_train)
```

LinearDiscriminantAnalysis()

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```
acc_score = accuracy_score(y_test, weekly_lda.predict(X_test))
```

```
print(f'Accuracy: {acc_score}')
```

```
print(f'Test Error: {1-acc_score}')
```

Accuracy: 0.625

Test Error: 0.375

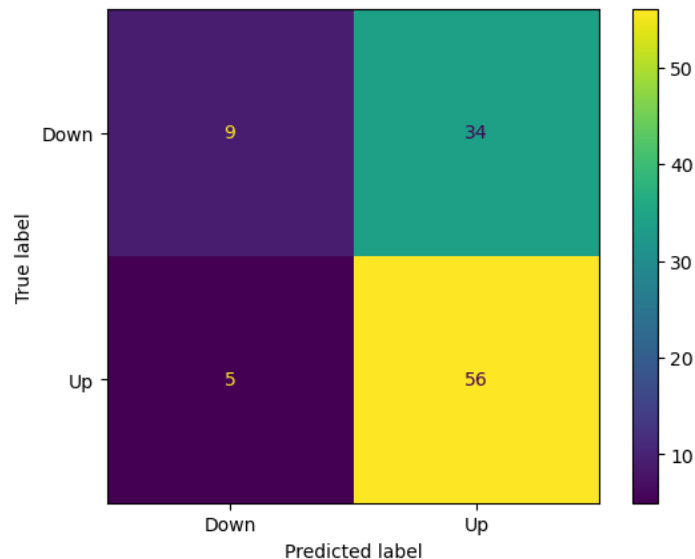
The model correctly predicts the market direction 62.5% of the time , which is better than random guessing (50%) . This means Lag2 can be predictive given the right data.

```
disp = ConfusionMatrixDisplay(confusion_matrix(y_test, weekly_lda.predict(X_test)),
```

```
display_labels=['Down', 'Up'])
```

```
disp.plot()
```

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1d131153750>



The same problem still exists in this model as well, as the model is still biased towards predicting an "Up" outcome.

(f) Repeat (d) using QDA.

```
weekly_qda = QuadraticDiscriminantAnalysis()
```

```
weekly_qda.fit(X_train, y_train)
```

```
QuadraticDiscriminantAnalysis()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```
acc_score = accuracy_score(y_test, weekly_qda.predict(X_test))
```

```
print(f'Accuracy: {acc_score}')
```

```
print(f'Test Error: {1-acc_score}')
```

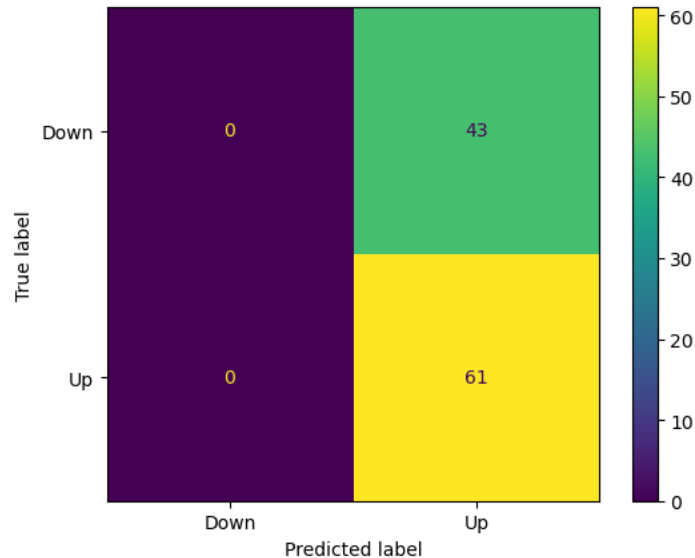
Accuracy: 0.5865384615384616

Test Error: 0.41346153846153844

```
disp = ConfusionMatrixDisplay(confusion_matrix(y_test, weekly_qda.predict(X_test)),  
display_labels=['Down', 'Up'])
```

```
disp.plot()
```

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1d12b75aea0>



The QDA model is performing worse than LDA on this data. Its accuracy rate is 59%, which is almost the same as the baseline performance rate. This suggests that the QDA model is predicting **Up** for every single test example, indicating a biased output.

(g) Repeat (d) using KNN with $K = 1$.

```
weekly_knn = KNeighborsClassifier(n_neighbors=1)
```

```
weekly_knn.fit(X_train, y_train)
```

```
KNeighborsClassifier(n_neighbors=1)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```
acc_score = accuracy_score(y_test, weekly_knn.predict(X_test))
```

```
print(f'Accuracy: {acc_score}')
```

```
print(f'Test Error: {1-acc_score}')
```

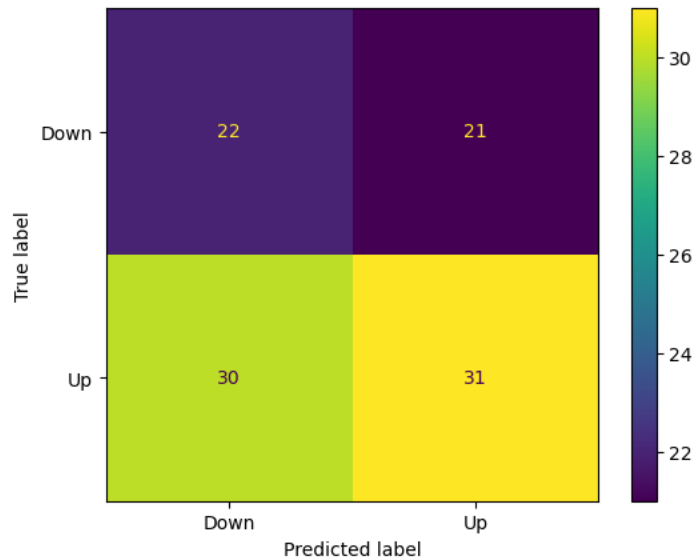
Accuracy: 0.5096153846153846

Test Error: 0.4903846153846154

```
disp = ConfusionMatrixDisplay(confusion_matrix(y_test, weekly_knn.predict(X_test)),  
display_labels=['Down', 'Up'])
```

```
disp.plot()
```

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1d12b75b230>



The KNN model is performing worse than the baseline model, which predicts Up for every test example. The accuracy rate of the KNN model is only 50%, which is not much better than random guessing.

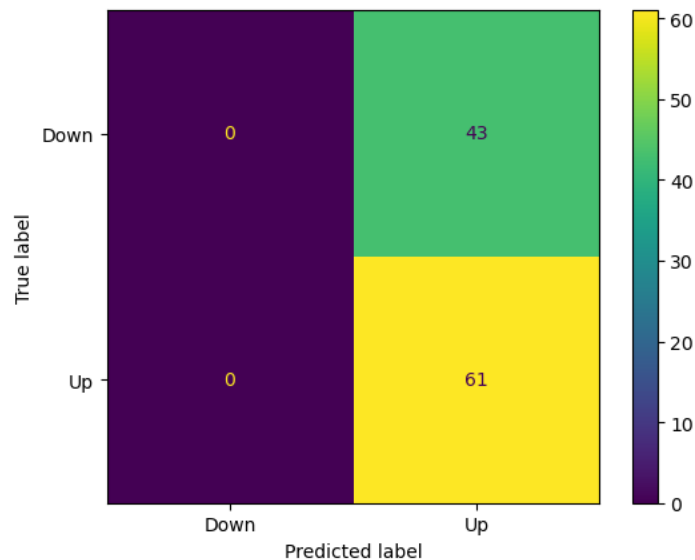
(h) Repeat (d) using naive Bayes.

```
weekly_naive_bayes = GaussianNB()
weekly_naive_bayes.fit(X_train, y_train)
GaussianNB()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```
acc_score = accuracy_score(y_test, weekly_naive_bayes.predict(X_test))
print(f'Accuracy: {acc_score}')
print(f'Test Error: {1-acc_score}')
Accuracy: 0.5865384615384616
Test Error: 0.41346153846153844
disp = ConfusionMatrixDisplay(confusion_matrix(y_test, weekly_naive_bayes.predict(X_test)),
display_labels=['Down', 'Up'])
disp.plot()
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1d12c3d0dd0>
```



Naive Bayes Method is not much better than the baseline method, we get an accuracy of 59%.

(i) Which of these methods appears to provide the best results on this data? Logistic Regression, and LDA giving the best accuracy.

(j) Experiment with different combinations of predictors, including possible transformations and interactions, for each of the methods. Report the variables, method, and associated confusion matrix that appears to provide the best results on the held out data. Note that you should also experiment with values for K in the KNN classifier.

results = []

```
def eval_preds(name, y_true, y_pred):
    cm = confusion_matrix(y_true, y_pred, labels=['Down','Up']) if y_pred.dtype.kind in
    {'U','S','O'} else confusion_matrix(y_true, y_pred, labels=[0,1])
    acc = accuracy_score(y_true, y_pred)
    results.append({'Model': name, 'Accuracy': acc, 'ConfusionMatrix': cm})
```

Helper to run statsmodels Logit given a patsy formula on train, then predict on test

```
def run_logit(formula):
    model = sm.Logit(tr['Direction_bin'],
    sm.add_constant(pd.DataFrame(sm.add_constant(pd.get_dummies(tr[['Direction']],
    drop_first=True)), index=train.index, columns=['const']) if False else
    sm.add_constant(train.eval(formula.split('~')[1]))))
    # The above is messy to accommodate patsy-less usage; instead use smf.logit for clarity:
    model = smf.logit('Direction_bin ~ ' + formula, data=train).fit(dispatch=False)
    proba = model.predict(ts)
    pred = (proba > 0.5).astype(int)
    eval_preds('Logit: ' + formula, ts['Direction_bin'], pred)
```

Candidate formulas (predictor side only)

```
formulas = [
    'Lag2',
```

```

'Lag1 + Lag2',
'Lag2 + Volume',
'Lag1 * Lag2',          # includes interaction
'Lag1 + Lag2 + Lag3 + Lag4 + Lag5 + Volume'
]

# Add polynomial Lag2 (degree 2) via explicit column
tr['Lag2_sq'] = tr['Lag2']**2
ts['Lag2_sq'] = ts['Lag2']**2
formulas.append('Lag2 + Lag2_sq')

# Evaluate Logit, LDA, QDA on these formulas
for rhs in formulas:
    # Logit
    try:
        run_logit(rhs)
    except Exception as e:
        pass

    # LDA/QDA need X as numeric matrix
    cols = [c.strip() for c in rhs.replace('*', '+').split('+')] # expand interaction as separate cols
    approx
    cols = [c for c in cols if c]
    Xtr = tr[cols].values
    Xte = ts[cols].values

    # Standardize using training stats (good for KNN but fine for LDA/QDA too)
    scaler = StandardScaler().fit(Xtr)
    Ztr, Zte = scaler.transform(Xtr), scaler.transform(Xte)

    try:
        lda_m = LinearDiscriminantAnalysis().fit(Ztr, tr['Direction'])
        pred = lda_m.predict(Zte)
        eval_preds('LDA: ' + rhs, ts['Direction'], pred)
    except Exception:
        pass

    try:
        qda_m = QuadraticDiscriminantAnalysis().fit(Ztr, tr['Direction'])
        pred = qda_m.predict(Zte)
        eval_preds('QDA: ' + rhs, ts['Direction'], pred)
    except Exception:
        pass

# KNN — multiple predictor sets & k values
knn_sets = {

```

```

'Lag2': ['Lag2'],
'Lag1+Lag2': ['Lag1','Lag2'],
'Lags1-5': ['Lag1','Lag2','Lag3','Lag4','Lag5'],
'Lags1-5+Volume': ['Lag1','Lag2','Lag3','Lag4','Lag5','Volume']
}
k_grid = [1,3,5,7,9,11,15,21]
for name, cols in knn_sets.items():
    Xtr = tr[cols].values
    Xte = ts[cols].values
    scaler = StandardScaler().fit(Xtr)
    Ztr, Zte = scaler.transform(Xtr), scaler.transform(Xte)
    for k in k_grid:
        knn = KNeighborsClassifier(n_neighbors=k)
        knn.fit(Ztr, tr['Direction'])
        pred = knn.predict(Zte)
        eval_preds(f'KNN(k={k}) [{name}]', ts['Direction'], pred)

```

```

res = pd.DataFrame(results).sort_values('Accuracy', ascending=False).reset_index(drop=True)
res.head(15)

```

C:\Users\Acer\AppData\Local\Temp\ipykernel_11588\1330098643.py:27: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
tr['Lag2_sq'] = tr['Lag2']**2
```

C:\Users\Acer\AppData\Local\Temp\ipykernel_11588\1330098643.py:28: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
ts['Lag2_sq'] = ts['Lag2']**2
```

	Model	Accuracy	ConfusionMatrix
0	LDA: Lag2	0.625000	[[9, 34], [5, 56]]
1	LDA: Lag2 + Lag2_sq	0.615385	[[7, 36], [4, 57]]
2	KNN(k=15) [Lag2]	0.586538	[[20, 23], [20, 41]]

	Model	Accuracy	ConfusionMatrix
3	LDA: Lag1 + Lag2	0.576923	[[7, 36], [8, 53]]
4	KNN(k=21) [Lag1+Lag2]	0.576923	[[21, 22], [22, 39]]
5	KNN(k=15) [Lag1+Lag2]	0.576923	[[21, 22], [22, 39]]
6	LDA: Lag1 * Lag2	0.576923	[[7, 36], [8, 53]]
7	KNN(k=3) [Lag2]	0.557692	[[16, 27], [19, 42]]
8	KNN(k=7) [Lag2]	0.557692	[[16, 27], [19, 42]]
9	KNN(k=9) [Lags1-5+Volume]	0.557692	[[23, 20], [26, 35]]
10	KNN(k=11) [Lag2]	0.557692	[[18, 25], [21, 40]]
11	KNN(k=21) [Lag2]	0.557692	[[19, 24], [22, 39]]
12	KNN(k=7) [Lags1-5]	0.557692	[[19, 24], [22, 39]]
13	KNN(k=3) [Lags1-5]	0.557692	[[21, 22], [24, 37]]
14	KNN(k=15) [Lags1-5+Volume]	0.557692	[[23, 20], [26, 35]]

```

formulas = [
    "Direction ~ Lag2",
    "Direction ~ Lag1 + Lag2",
    "Direction ~ Lag2 + Volume",
    "Direction ~ Lag2 + I(Lag2**2)",
    "Direction ~ Lag1 * Lag2",
    "Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 + Volume"
]
# Show the single best model and its confusion matrix
best_row = res.iloc[0]
best_row

```

```
Model          LDA: Lag2
Accuracy        0.625
ConfusionMatrix [[9, 34], [5, 56]]
Name: 0, dtype: object
print('Best model:', best_row['Model'])
print('Best accuracy:', best_row['Accuracy'])
Best model: LDA: Lag2
Best accuracy: 0.625
```